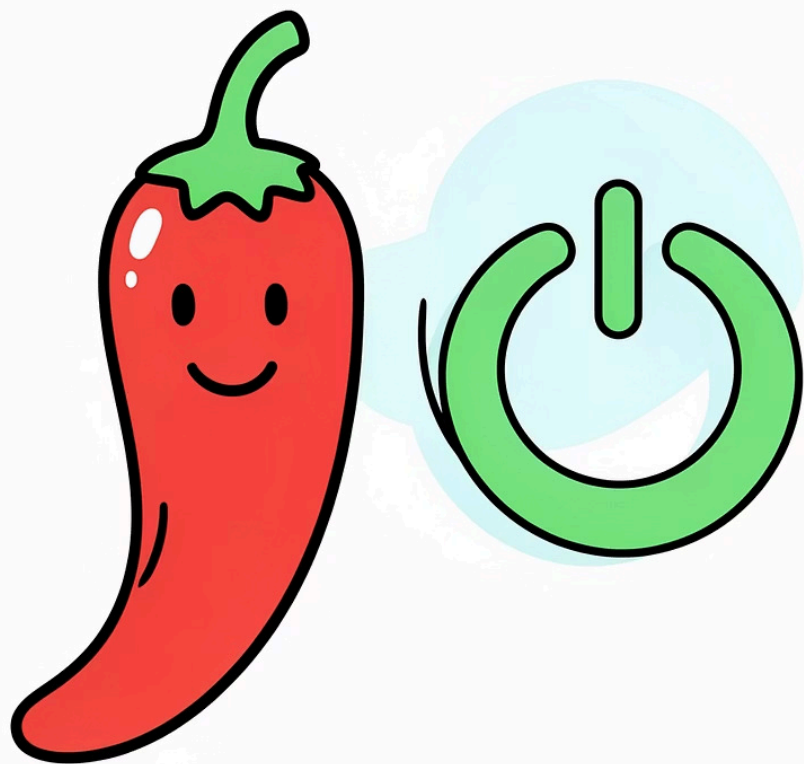


PROGRAMACIÓN III

Lombok y DTO

En esta presentación veremos cómo estas herramientas nos permiten reducir código repetitivo, mejorar la claridad del proyecto y aplicar buenas prácticas en el desarrollo.





¿Qué es Lombok?

Lombok es una biblioteca de Java que reduce drásticamente el código repetitivo mediante anotaciones que generan código automáticamente en tiempo de compilación.

Se integra perfectamente con el ecosistema Java, siendo especialmente popular en proyectos Spring Boot, microservicios y APIs REST.

¿Por qué usar Lombok?

Ahorro de tiempo

Elimina la necesidad de escribir getters, setters, constructores y métodos toString()

Mayor legibilidad

Reduce hasta un 40% las líneas de código, permitiendo enfocarse en la lógica del negocio

Menos errores

Al generar código automáticamente, se reducen los errores humanos en métodos estándar

📌 **Nota importante:** "Lombok es una herramienta para hacerte más productivo cuando ya comprendes los conceptos fundamentales."

Configuración de Lombok

Maven

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.30</version>
  <scope>provided</scope>
</dependency>
```

Gradle

```
compileOnly 'org.projectlombok:lombok:1.18.30'
annotationProcessor 'org.projectlombok:lombok:1.18.30'
```

Configuración del Proyecto

- Utilizar Maven o Gradle como gestor de dependencias.
- Agregar la dependencia en:
 - pom.xml (si se usa Maven)
 - build.gradle (si se usa Gradle)
- Actualizar dependencias
- Instalar el plugin de Lombok (Eclipse o IntelliJ IDEA).
- Verificar que el proyecto compile correctamente.

Anotaciones de Lombok



@Getter / @Setter

Genera métodos getter y setter automáticamente



@ToString

Genera método toString() personalizable



@EqualsAndHashCode

Genera equals() y hashCode() basados en campos



@Builder

Genera patrón builder completo para construcción fluida



@Data

Combinación de @Getter, @Setter, @ToString, @EqualsAndHashCode



@NoArgsConstructor

Genera un constructor sin argumentos



@AllArgsConstructor

Genera un constructor con todos los campos como parámetros

Buenas prácticas con Lombok

Especificidad sobre conveniencia

Prefiere anotaciones específicas (@Getter, @ToString) sobre anotaciones combinadas (@Data) cuando necesites control granular

Uso adecuado de @Data

Usa @Data solo en clases que representen estructuras de datos simples (DTOs, modelos básicos)

Separación de responsabilidades

Evita mezclar lógica compleja con Lombok, especialmente en entidades JPA con relaciones complejas

Documentación

Incluye comentarios que indiquen qué anotaciones estás usando, especialmente para otros desarrolladores menos familiarizados con Lombok

Antes y después

Sin Lombok (70+ líneas)

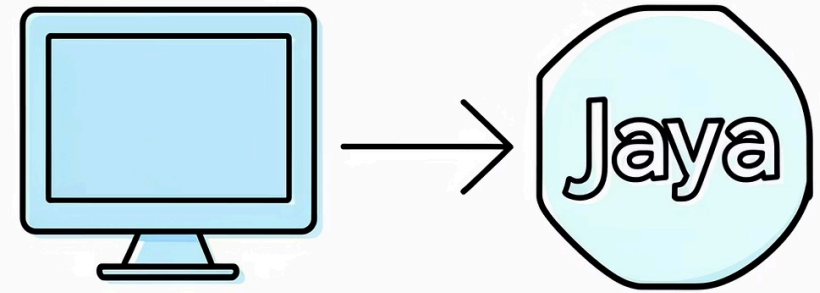
```
public class Usuario {  
    private Long id;  
    private String nombre;  
    private String email;  
    private String password;  
    private int edad;  
  
    public Usuario() {  
    }  
  
    public Usuario(Long id, String nombre, String email,  
String password, int edad) {  
        // ...  
    }  
  
    // Getters y setters (20+ líneas)  
  
    // equals(), hashCode(), toString() (15+ líneas)  
}
```

Con Lombok (10 líneas)

```
import lombok.AllArgsConstructor;  
import lombok.Builder;  
import lombok.Data;  
import lombok.NoArgsConstructor;  
  
@Data  
@Builder  
@NoArgsConstructor  
@AllArgsConstructor  
public class Usuario {  
    private Long id;  
    private String nombre;  
    private String email;  
    private String password;  
    private int edad;  
}
```

DTOs en Java:

Buenas prácticas y modernización con records



¿Qué es un DTO (Data Transfer Object)?

Un Data Transfer Object es un patrón de diseño que define objetos simples para transportar datos entre subsistemas de una aplicación, especialmente a través de límites de red o entre capas.

Los DTOs se centran exclusivamente en la transferencia de datos, sin incluir comportamiento de negocio o validaciones complejas.

Propósito Principal

Transportar datos entre capas o procesos sin exponer la lógica interna del dominio, actuando como contenedores simples de información.

Características

Objetos serializables, generalmente inmutables, y desprovistos de comportamiento complejo o lógica de negocio.

¿Por qué usar DTOs?

Separación de Capas

Evita la exposición directa de entidades de dominio, estableciendo límites claros entre capas de la aplicación y mejorando el encapsulamiento.

Seguridad

Previene la filtración de datos sensibles al controlar exactamente qué información se transmite, evitando exponer atributos internos o confidenciales.

Control de Datos

Permite seleccionar precisamente qué datos enviar/recibir, optimizando transferencias y adaptando interfaces a necesidades específicas.

Reducción del Acoplamiento

Minimiza la dependencia entre capas, facilitando cambios en el modelo de dominio sin afectar las interfaces externas.

- ❏ Los DTOs son especialmente valiosos en arquitecturas distribuidas, APIs REST y aplicaciones multicapa donde la transferencia eficiente y segura de datos es crítica.

Entity vs DTO

Usar entidades directamente en controladores o interfaces externas crea problemas significativos de seguridad, rendimiento y mantenibilidad. Es esencial entender las diferencias fundamentales entre estos componentes.

Aspecto	Entity	DTO
Propósito	Representar y persistir datos del dominio	Transferir datos entre capas o sistemas
Responsabilidad	Implementar lógica de dominio y reglas de negocio	Transportar datos sin comportamiento adicional
Relaciones	Puede tener relaciones complejas y bidireccionales	Relaciones simplificadas o referencias por ID
Persistencia	Anotado para JPA/Hibernate y persistencia	Sin anotaciones de persistencia
Exposición	Solo interno, no debe exponerse directamente	Diseñado específicamente para exposición externa
Mutabilidad	Generalmente mutable	Generalmente inmutable

DTO clásico (POJO)

Características del DTO clásico:

- Campos privados para encapsulamiento
- Getters y setters para cada atributo
- Constructor(es) con parámetros
- Implementaciones de equals(), hashCode(), toString()
- A veces, implementación de Serializable

Código de ejemplo:

```
public class ProductDTO {  
    private Long id;  
    private String name;  
    private double price;  
  
    public ProductDTO(Long id, String name, double price) {  
        // Inicialización de campos  
    }  
  
    // Getters y setters (20+ líneas)  
  
    // equals(), hashCode(), toString() (15+ líneas)  
}
```

Este enfoque tradicional genera boilerplate excesivo, dificultando la lectura y mantenimiento, especialmente para DTOs con muchos atributos.

Introducción a record (Java 14+)

Desde Java 14, el tipo `record` ofrece una forma concisa de definir clases inmutables de transferencia de datos, eliminando prácticamente todo el boilerplate asociado con los DTOs tradicionales.



Declaración Compacta

Define campos, constructor y métodos de acceso en una sola línea



Inmutabilidad

Todos los campos son `final` por defecto, garantizando objetos inmutables



Generación Automática

Implementaciones automáticas de `equals()`, `hashCode()`, `toString()` y getters

```
// DTO record
public record UserDTO(Long id, String username, String email) {}

// Crear el DTO
UserDTO usuario = new UserDTO(1L, "pablo_dev", "pablo@example.com");
```

Limitaciones de record

Aunque los records son muy convenientes para DTOs, tienen limitaciones importantes que debes considerar antes de adoptarlos universalmente.

Sin herencia de clases

Los records no pueden extender otras clases y son implícitamente finales. Solo pueden implementar interfaces, limitando la jerarquía de DTOs.

Campos siempre finales

Todos los campos son inmutables, lo que puede ser inconveniente cuando necesitas modificar propiedades después de la creación.

Compatibilidad con frameworks

Algunos frameworks antiguos o bibliotecas que dependen de JavaBeans pueden no funcionar correctamente con records (JPA, ciertos serializadores).

Personalización limitada

Menor flexibilidad para personalizar la generación de métodos estándar como `equals()`, `hashCode()` o `accessors`.

Validación en DTOs

Los DTOs son clave para validar y asegurar la integridad de los datos de entrada, especialmente en APIs públicas.



@NotBlank / @NotNull

Aseguran que campos esenciales no estén vacíos o nulos, garantizando datos críticos.



@Size / @Length

Definen límites mínimos y máximos para la longitud de textos o colecciones.



@Email / @Pattern

Valida formatos específicos, como correos electrónicos o expresiones regulares.



@Min / @Max

Establecen valores mínimos y máximos para campos numéricos dentro de un rango aceptable.

Tipos de DTOs comunes

RequestDTO

Captura datos de entrada del cliente hacia el servidor

ResponseDTO

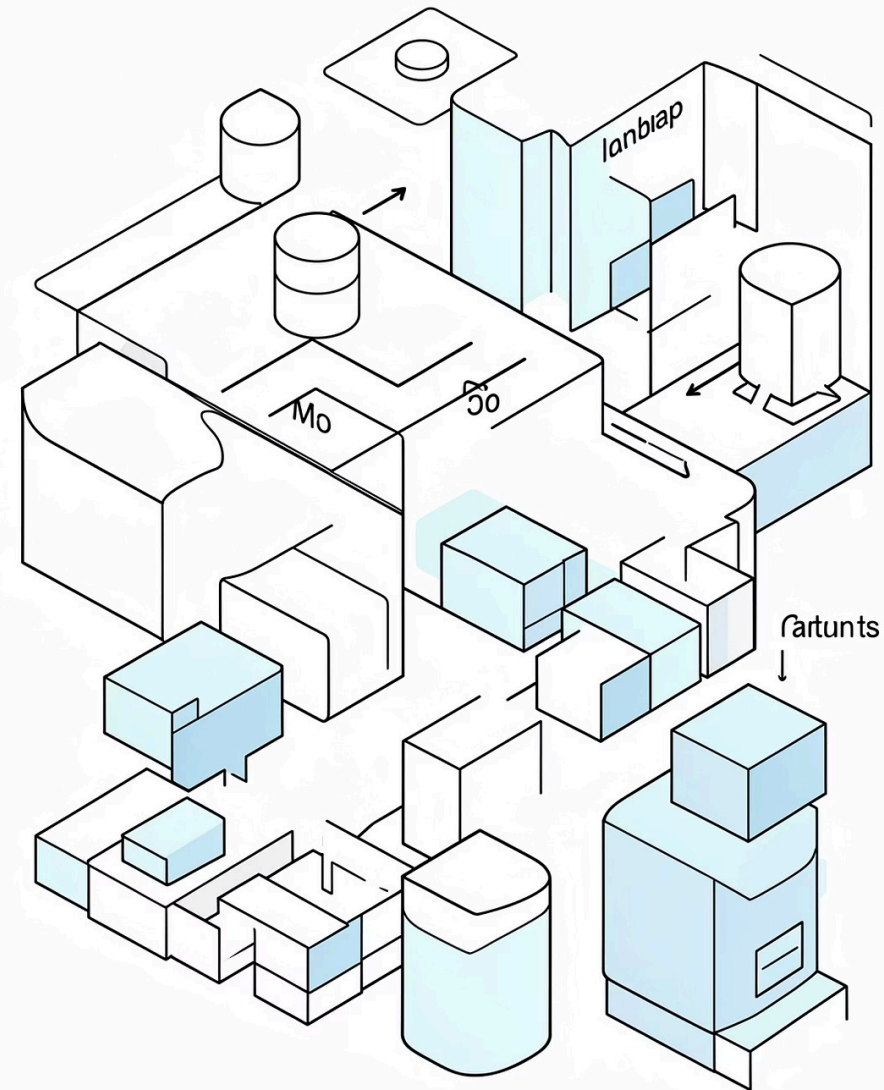
Formatea datos de salida del servidor hacia el cliente

PageDTO

Encapsula resultados paginados para consultas grandes

ErrorDTO

Estructura mensajes de error de forma consistente



Maapeo entre Entity y DTO

El mapeo entre entidades de dominio y DTOs es una operación frecuente en aplicaciones bien estructuradas. Existen varias aproximaciones para realizarlo, desde mapeo manual hasta el uso de bibliotecas especializadas.

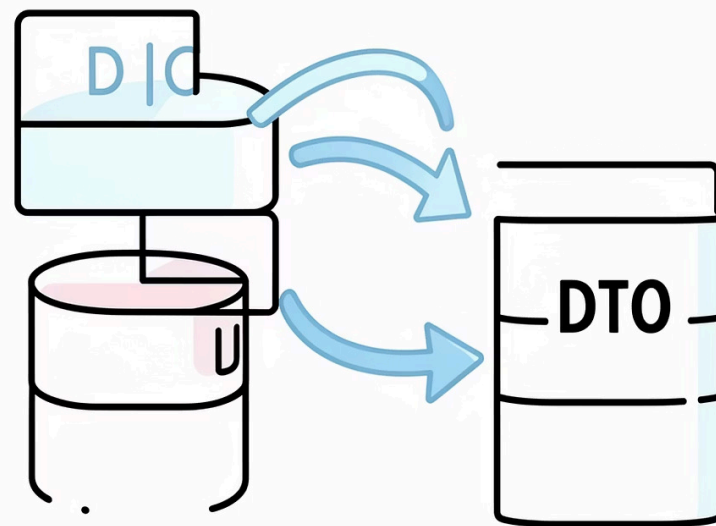
Dos enfoques principales:

Mapeo manual

Escribir métodos específicos para convertir entre Entity y DTO. Simple y explícito, pero repetitivo para objetos complejos.

Bibliotecas de mapeo

Herramientas como MapStruct o ModelMapper automatizan la conversión, reduciendo código repetitivo y errores.



Estructura típica en una API REST

Los DTOs son componentes esenciales en una arquitectura de API REST bien diseñada, actuando como intermediarios entre las capas y protegiendo la integridad del dominio.



Request al Controller



Convertir a DTO



Procesar en Service



Interacción con Repository

Beneficios de esta estructura:

- Clara separación de responsabilidades entre capas
- Protección del modelo de dominio
- Facilita cambios independientes en cada capa
- Control sobre los contratos de API

Buenas prácticas con DTOs

1

Proteger el modelo de dominio

Nunca exponer entidades directamente a través de APIs. Usar siempre DTOs como intermediarios para controlar exactamente qué datos se transfieren y evitar vulnerabilidades de seguridad.

2

DTOs específicos por caso de uso

Diseñar DTOs distintos para cada operación en lugar de reutilizar un DTO genérico. Esto permite adaptarlos a las necesidades exactas de cada endpoint o función.

3

Separar DTOs de entrada y salida

Mantener DTOs diferentes para requests y responses, incluso cuando parezcan similares, permite evolucionar independientemente los contratos de entrada y salida.

4

Versionar DTOs cuando sea necesario

Para APIs públicas, considerar versionado de DTOs para permitir cambios sin romper compatibilidad con clientes existentes.

- ❏ Recuerda que los DTOs son contratos de comunicación que definen las interfaces de tu aplicación. Diseñarlos cuidadosamente y mantenerlos actualizados es clave para un sistema robusto y flexible.

Comparación: DTO POJO vs Lombok vs Record

DTO POJO (Tradicional)

```
public class ProductDTO {  
    private Long id;  
    private String name;  
    private double price;  
  
    public ProductDTO(Long id,  
String name, double price) {  
        this.id = id;  
        this.name = name;  
        this.price = price;  
    }  
  
    // Getters y setters (10+ líneas)  
    // equals(), hashCode(),  
    toString() (15+ líneas)  
}
```

Total: ~50 líneas

DTO con Lombok

```
@Data  
@NoArgsConstructor  
@AllArgsConstructor  
public class ProductDTO {  
    private Long id;  
    private String name;  
    private double price;  
}
```

Total: ~10 líneas

DTO Record (Java 14+)

```
public record ProductDTO(  
    Long id,  
    String name,  
    double price  
) {}
```

Total: ~4 líneas

❏ Los records ofrecen la solución más concisa y moderna para DTOs inmutables

¡Gracias!

Esperamos que esta presentación te haya proporcionado una comprensión clara de cómo usar Lombok y DTOs para mejorar tu desarrollo en Java.

¡Hasta la próxima!

